

Clasificador de Componentes Electrónicos en Tiempo Real

Proyecto Final – Visión Artificial

Agostini, Emiliano S.
Cortesse, Agustín

Departamento de Ingeniería Electrónica
Facultad Regional San Francisco, Córdoba, Argentina

29 de Junio de 2026

Contenido

- 1 Resumen
- 2 Introducción
- 3 Herramientas utilizadas
- 4 Descripción del sistema
- 5 Pruebas del sistema
- 6 Conclusiones

Resumen del proyecto

- Sistema de visión artificial que identifica componentes electrónicos en **tiempo real** mediante cámara web.
- Cuatro clases de componentes:
 - Resistencias
 - LED
 - Capacitores electrolíticos
 - Capacitores cerámicos
- Arquitectura base: **MobileNetV2** adaptado mediante *transfer learning*.
- Entrenamiento con **PyTorch**, captura e inferencia con **OpenCV**.
- Resultado: clasificación en vivo con porcentaje de confianza, ejecutado completamente sobre CPU.

Objetivos del proyecto

Objetivo general: desarrollar un sistema capaz de identificar componentes electrónicos mediante visión artificial, utilizando un modelo de aprendizaje profundo.

Objetivos específicos:

- Construir un conjunto de imágenes propio para el entrenamiento.
- Entrenar un modelo de clasificación utilizando PyTorch.
- Implementar un sistema de reconocimiento en tiempo real mediante una cámara web.
- Evaluar el desempeño del modelo obtenido.

Herramientas utilizadas

Desarrollo realizado en **Windows**, con **VS Code** como IDE y un entorno virtual de Python (venv).

Herramienta	Versión	Función
Python	3.14	Lenguaje de desarrollo del proyecto
PyTorch	2.12.0 (CPU)	Entrenamiento mediante transfer learning
Torchvision	—	Modelo MobileNetV2 preentrenado + transformaciones
OpenCV	4.13.0	Captura de video, recorte interactivo, overlay
PIL / NumPy	—	Conversión de color y operaciones numéricas

Arquitectura general del sistema

El sistema se divide en tres etapas independientes, cada una implementada como un script separado:

Archivo	Función
<code>dataset.py</code>	Organiza las imágenes y crea train/val
<code>train.py</code>	Entrena el modelo y guarda el <code>.pth</code>
<code>inference.py</code>	Clasifica en tiempo real con la cámara web

Ventaja de esta separación: el modelo se reutiliza sin reentrenar, y cada etapa puede modificarse de forma aislada (pipeline de datos / entrenamiento / servicio).

Captura del conjunto de imágenes

- Fotografías propias.
- Fondo blanco homogéneo → mayor contraste y menor ruido visual.
- 3 vistas por componente: lateral, 45 y superior.
- 10–15 rotaciones por vista → variabilidad angular.

```
data/raw/  
|-- resistencia/  
|-- led/  
|-- capacitor_electrolitico/  
|-- capacitor_ceramico/
```

Por qué importa la variabilidad: un dataset reducido o poco variado produce *overfitting* – el modelo memoriza en lugar de generalizar.

Problema detectado: componentes ocupando una porción muy reducida del encuadre
→ al redimensionar a 224×224 px, se pierde información visual relevante.

Solución: herramienta de recorte (`manual_crop.py`)

- Se selecciona la región de interés con el mouse.
- Se guarda el recorte en una carpeta separada.
- Resultado: mayor proporción del componente en el encuadre.
- Mejor confianza de predicción.

Ejemplo: capturas del capacitor electrolítico



Vista lateral



Vista a 45°



Vista superior

Tres ángulos distintos del mismo componente, capturados sobre el mismo fondo blanco, con rotaciones adicionales sobre cada eje.

División del dataset: train / val

`dataset.py` separa las imágenes en **80 % entrenamiento** y **20 % validación**, con semilla fija para reproducibilidad.

Clase	Entrenamiento	Validación
Resistencia	226	57
LED	213	54
Capacitor electrolítico	272	68
Capacitor cerámico	192	49
Total	903	228

El conjunto de validación nunca es visto durante el entrenamiento → mide la capacidad real de generalización del modelo.

¿Qué es MobileNetV2?

- Una red neuronal convolucional ya entrenada para ver e interpretar imágenes.
- Diseñada para ser **liviana y rápida**, ideal para correr en una computadora común, sin necesidad de placa de video.

¿Cómo aprovechamos eso? (*Transfer Learning*)

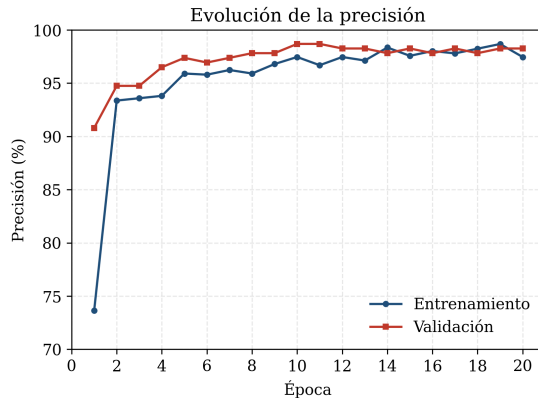
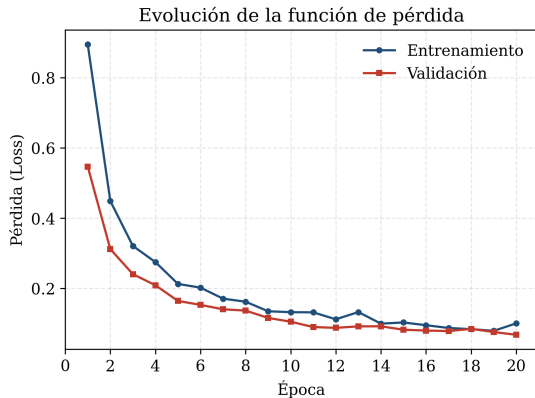
- En vez de entrenar una red desde cero, partimos de un modelo que ya aprendió a reconocer formas, bordes y texturas a partir de **más de un millón de imágenes**.
- Le "enseñamos" únicamente a distinguir nuestras 4 clases de componentes, reutilizando todo ese conocimiento previo.

Resultado: un modelo preciso, entrenado en pocos minutos, sin necesidad de miles de fotos ni hardware especializado.

Parámetros de entrenamiento

Parámetro	Valor
Modelo	MobileNetV2
Cantidad de clases	4
Épocas	20
Batch Size	16
Learning Rate	0.001
Tamaño de imagen	224×224 px
Optimizador	Adam
Función de pérdida	CrossEntropyLoss

Resultado del entrenamiento



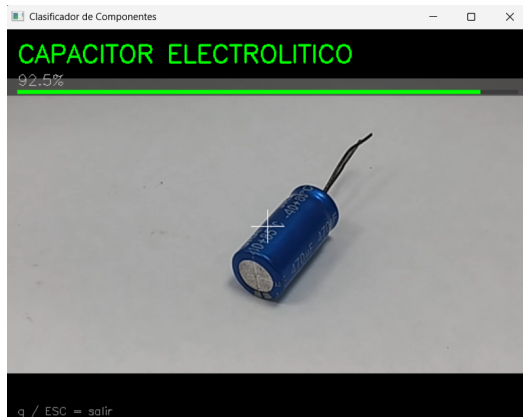
- Las curvas se estabilizan a partir de la época ~ 10 .
- Validación $\geq$ entrenamiento en todo momento $\rightarrow$ sin *overfitting*.
- Mejor **accuracy de validación: 98.68 %**.

Clasificación en tiempo real



- 1 Captura del fotograma (BGR $\rightarrow$ RGB).
- 2 Resize a 224×224 px + normalización.
- 3 Inferencia con MobileNetV2 $\rightarrow$ vector de probabilidades (softmax).
- 4 **Suavizado temporal** (promedio de los últimos 12 fotogramas).
- 5 Visualización del resultado en pantalla.

Pruebas del sistema



En pantalla se visualiza: nombre del componente, porcentaje de confianza, e imagen en tiempo real. Si la confianza no supera el **60 %**, se evita mostrar una clasificación incorrecta.

Problemas detectados y soluciones

- **Componente alejado del encuadre** → recorte interactivo manual.
- **Confianza de predicción baja** → consecuencia directa del punto anterior; se corrigió al recortar las imágenes.
- **Mezcla de variantes en una misma clase** (resistencias axiales + de potencia) → se depuró el dataset, conservando solo resistencias axiales pequeñas.
- **Oscilación de la predicción en vivo** → suavizado temporal con buffer de 12 fotogramas.

Este proceso iterativo de prueba y ajuste fue clave para alcanzar el desempeño final del sistema.

- Se desarrolló íntegramente un sistema de visión artificial funcional, desde la captura de datos hasta la clasificación en tiempo real.
- MobileNetV2 + transfer learning permitió un modelo liviano, preciso y ejecutable en CPU.
- Código modular (pipeline de datos / entrenamiento / servicio), fácilmente extensible.

Limitaciones actuales:

- Restringido a las 4 clases entrenadas.
- Sensible a iluminación extrema, distancia y oclusión parcial.

- Incorporar nuevas familias de componentes.
- Diseñar una interfaz gráfica más robusta con información técnica adicional por componente.
- Optimizar y desplegar el modelo en dispositivos embebidos (ej. Raspberry Pi).

¡Gracias!